GCI World

Feature Engineering

松尾・岩澤研究室
MATSUO-IWASAWA LAB UTOKYO

Photography, video recording and disclosure to third parties without permissions are strictly prohibited.

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Contents

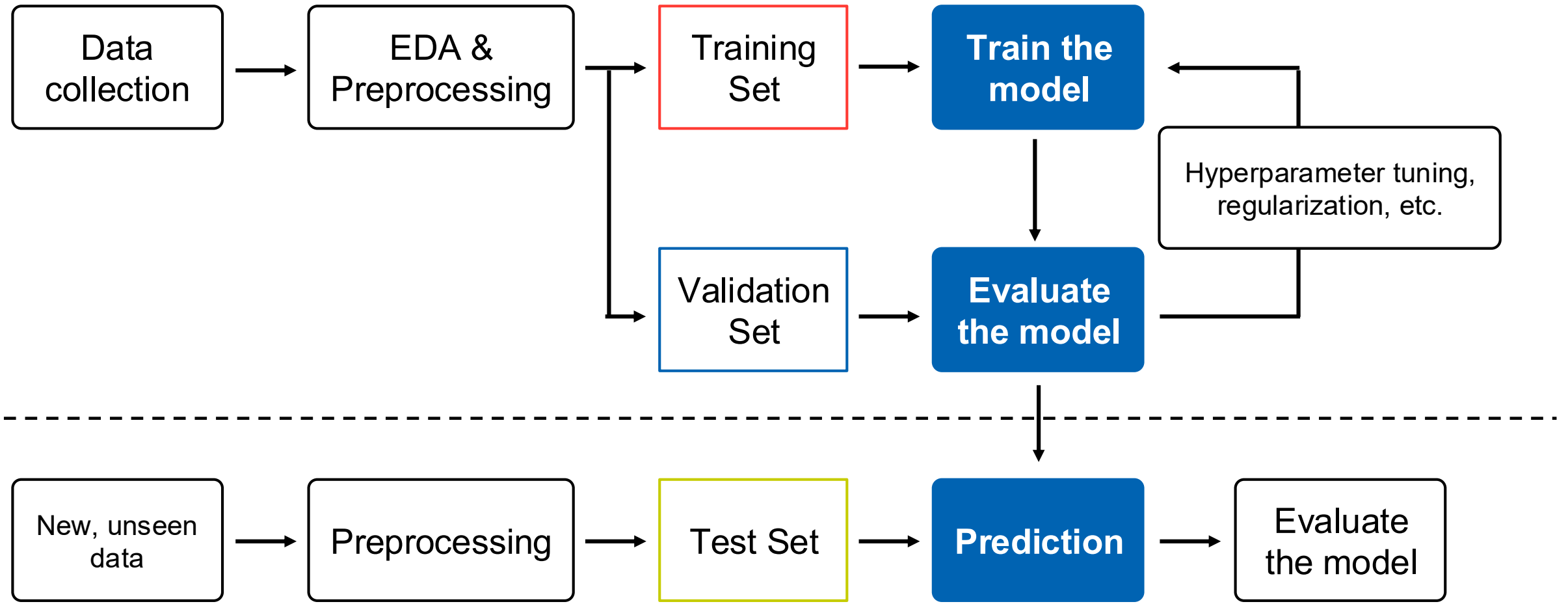
1. Overview of Feature Engineering
2. Transforming Numerical Variables
3. Transforming Categorical Variables
4. Use of Domain Knowledge
5. Ensembling

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Overview of Feature Engineering

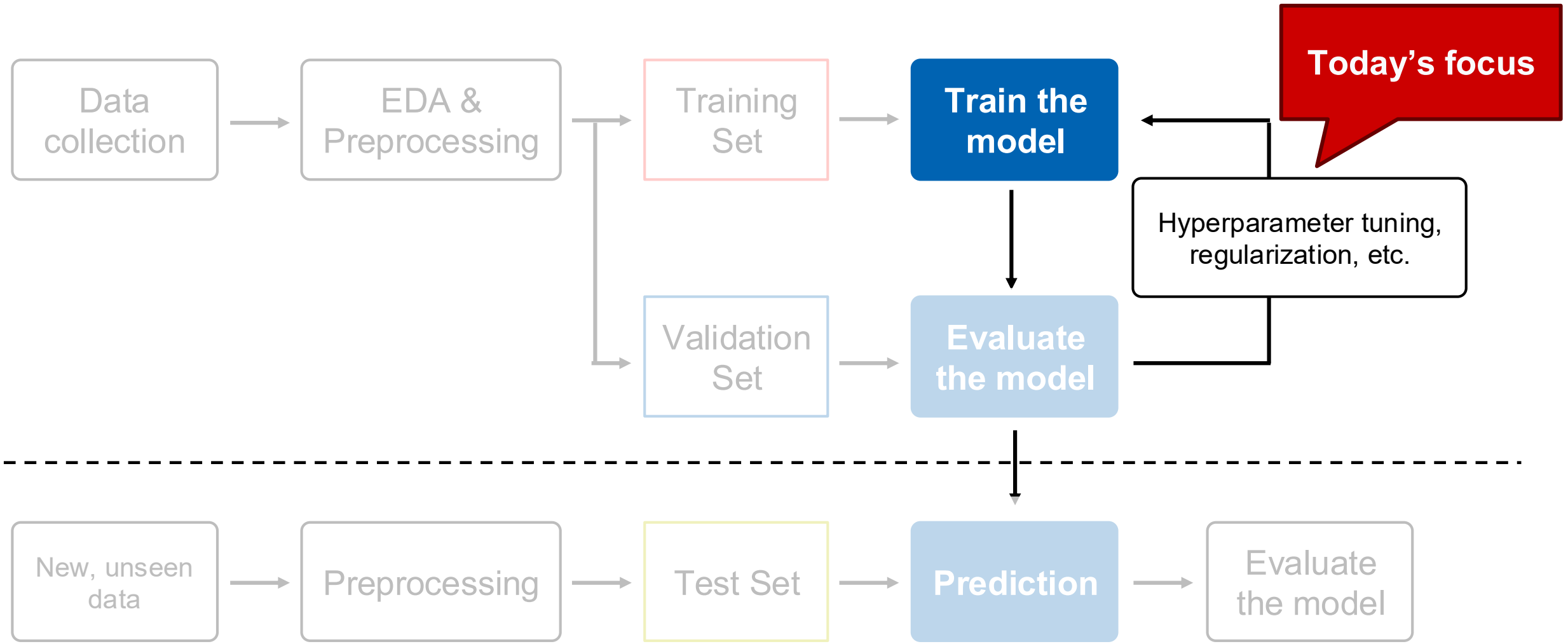
©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Supervised Learning Workflow (Summary)



©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Supervised Learning Workflow (Summary)



©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

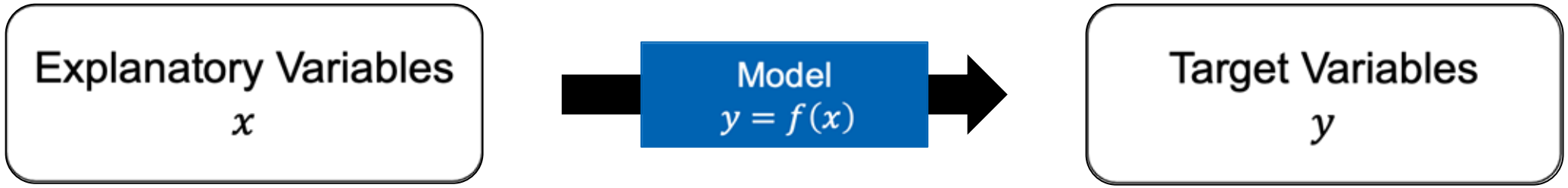
Feature Engineering

The process selecting, transforming, or creating input variables to improve the performance of a machine learning model

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Feature Engineering

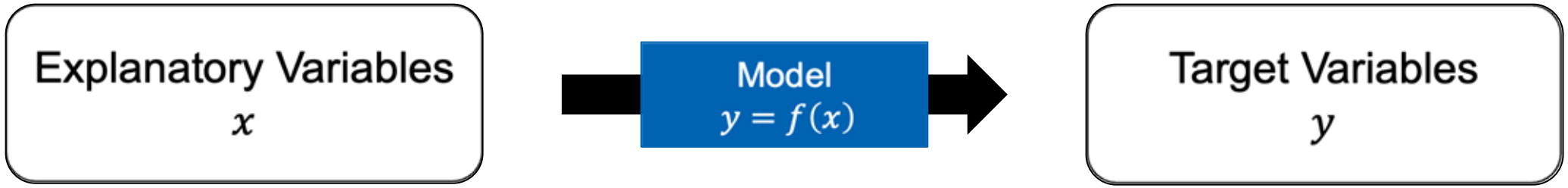
The process selecting, transforming, or creating input variables to improve the performance of a machine learning model



©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Feature Engineering

The process selecting, transforming, or creating input variables to improve the performance of a machine learning model

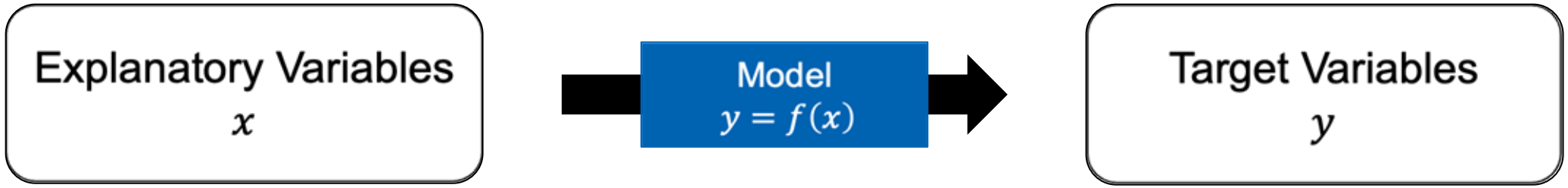


- e.g. Sales prediction
  - x: date, product ID
  - y: sales

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Feature Engineering

The process selecting, transforming, or creating input variables to improve the performance of a machine learning model



- e.g. Sales prediction
  - x: ~~date, product ID~~ → **day, holiday or not, season, product category**
  - y: sales

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Feature Engineering

Why is feature engineering so important?

- To boost model performance
- To prevent overfitting
- To make your model more interpretable

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Feature Engineering

Why is feature engineering so important?

- To boost model performance
- To prevent overfitting
- To make your model more interpretable

Feature engineering often requires trial and error.

- In this lecture, we will learn general techniques for designing useful features for different datasets.



©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Types of Feature Engineering

Transforming features

Creating features

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Types of Feature Engineering

Transforming features

- Transform the data for the model to handle inputs easier
  - Scaling, log-transformation
  - Encoding

Creating features

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Types of Feature Engineering

Transforming features

- Transform the data for the model to handle inputs easier
  - Scaling, log-transformation
  - Encoding

Creating features

- Make new features to improve model's performance
  - Cross/combine existing features
  - Use domain knowledge to create new features

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Types of Feature Engineering

Transforming features

- Transform the data for the model to handle inputs easier
  - Scaling, log-transformation
  - Encoding

Creating features

- Make new features to improve model's performance
  - Cross/combine existing features
  - Use domain knowledge to create new features

Extracting features

Selecting features

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Types of Feature Engineering

Transforming features

- Transform the data for the model to handle inputs easier
  - Scaling, log-transformation
  - Encoding

Creating features

- Make new features to improve model's performance
  - Cross/combine existing features
  - Use domain knowledge to create new features

Extracting features

- Make new features to improve model's performance
  - Cross/combine existing features
  - Use domain knowledge to create new features

Selecting features

- Remove features when that the input dimension is too large than data points
  - Feature selection

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Types of Feature Engineering

Transforming features

- Transform the data for the model to handle inputs easier
  - **Scaling, log-transformation**
  - **Encoding**

Creating features

- Make new features to improve model's performance
  - **Cross/combine existing features**
  - **Use domain knowledge to create new features**

Extracting features

- Make new features to improve model's performance
  - Cross/combine existing features
  - Use domain knowledge to create new features

Selecting features

- Remove features when that the input dimension is too large than data points
  - Feature selection

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming Numerical Variables and Categorical Variables

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Classifications of variables

Type	Description	Examples
Quantitative data (**Numerical variables**)		
Qualitative data (**Categorical variables**)		

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Classifications of variables

	Type	Description	Examples
Quantitative data (**Numerical variables**)	Ratio scale	Difference has meaning and has zero point	
Qualitative data (**Categorical variables**)			

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Classifications of variables

	Type	Description	Examples
Quantitative data (**Numerical variables**)	Ratio scale	Difference has meaning and has zero point	• Body weight (50 kg vs 70kg) • Distance (2km vs 200km)
Qualitative data (**Categorical variables**)			

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Classifications of variables

	Type	Description	Examples
Quantitative data (**Numerical variables**)	Ratio scale	Difference has meaning and has zero point	• Body weight (50 kg vs 70kg) • Distance (2km vs 200km)
	Interval scale	Difference has meaning	
Qualitative data (**Categorical variables**)			

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Classifications of variables

	Type	Description	Examples
Quantitative data (**Numerical variables**)	Ratio scale	Difference has meaning and has zero point	• Body weight (50 kg vs 70kg) • Distance (2km vs 200km)
	Interval scale	Difference has meaning	• Temperature (15℃, 30℃) • Year (2000, 2025)
Qualitative data (**Categorical variables**)			

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Classifications of variables

	Type	Description	Examples
Quantitative data (**Numerical variables**)	Ratio scale	Difference has meaning and has zero point	• Body weight (50 kg vs 70kg) • Distance (2km vs 200km)
	Interval scale	Difference has meaning	• Temperature (15℃, 30℃) • Year (2000, 2025)
Qualitative data (**Categorical variables**)	Ordinal scale	Order has meaning	
	Nominal scale	Distinct meanings	

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Classifications of variables

	Type	Description	Examples
Quantitative data (**Numerical variables**)	Ratio scale	Difference has meaning and has zero point	• Body weight (50 kg vs 70kg) • Distance (2km vs 200km)
	Interval scale	Difference has meaning	• Temperature (15℃, 30℃) • Year (2000, 2025)
Qualitative data (**Categorical variables**)	Ordinal scale	Order has meaning	• Academic score (A, B, C) • Questionnaire (good, bad)
	Nominal scale	Distinct meanings	• Nationality (Japanese, Chinese) • Product ID (42, 57)

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

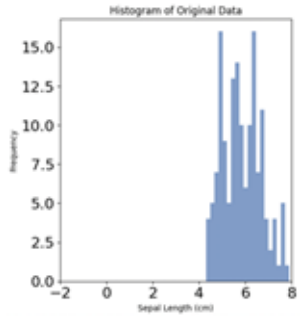
Transforming Numerical Variables

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming numeric features: Scaling

Scaling

If features differ greatly in scale, the larger scaled features can have an undue influence on the results

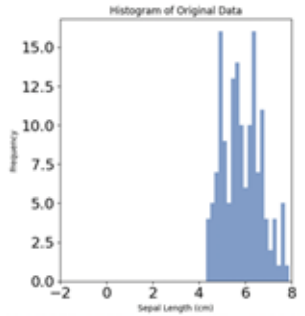


©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming numeric features: Scaling

Scaling

If features differ greatly in scale, the larger scaled features can have an undue influence on the results



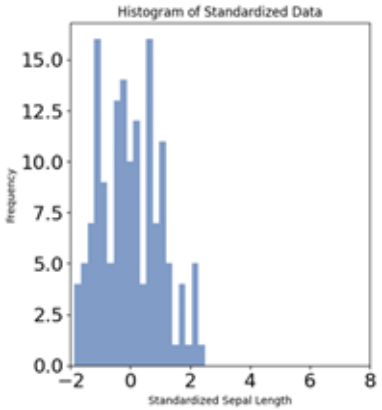
Standardization

Transform so that the mean is 0 and the standard deviation is 1

$$x_{std} = \frac{x - \mu_x}{\sigma_x}$$

Mean

Standard Deviation

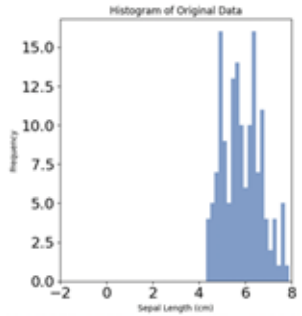


©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming numeric features: Scaling

Scaling

If features differ greatly in scale, the larger scaled features can have an undue influence on the results



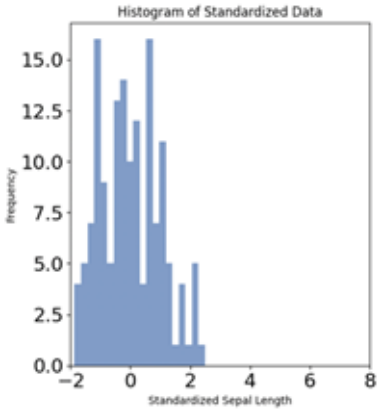
Standardization

Transform so that the mean is 0 and the standard deviation is 1

$$x_{std} = \frac{x - \mu_x}{\sigma_x}$$

Mean

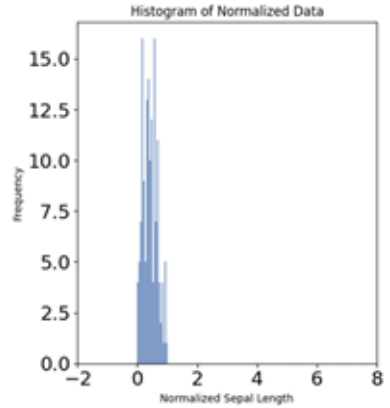
Standard Deviation



Normalization

Transform so that the minimum is 0 and the maximum is 1

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

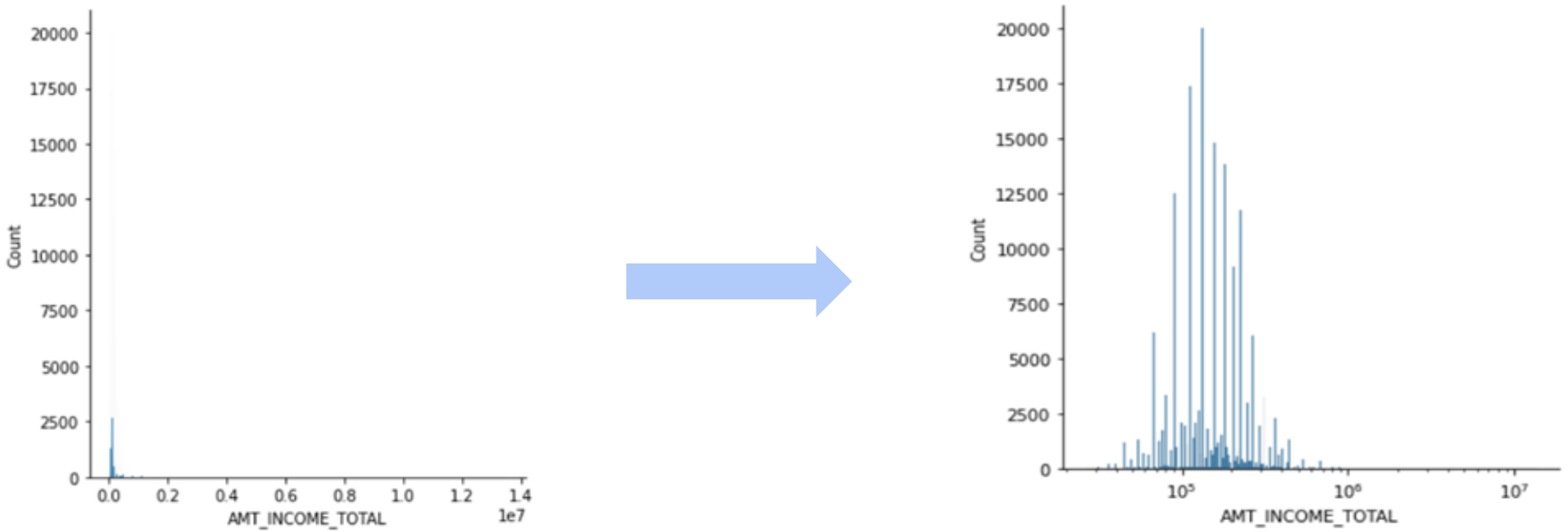


©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming numeric features: Log Transformation

Log transformation

- Compresses large values and makes relationships between variables easier for a model to capture

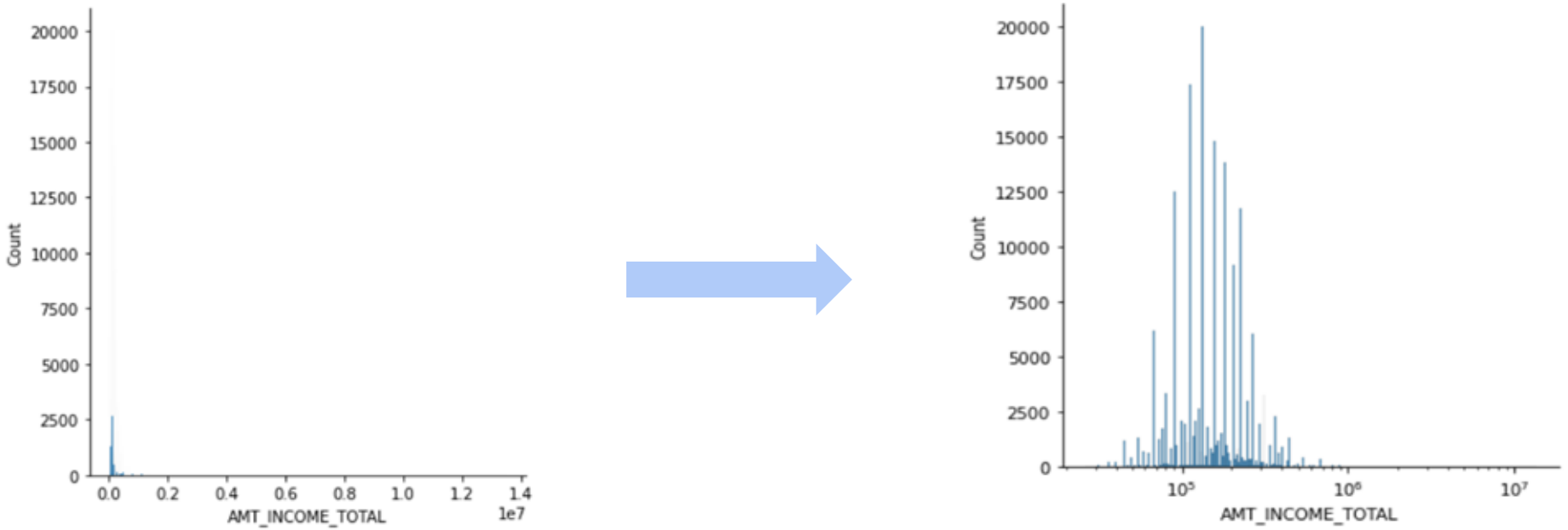


©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming numeric features: Log Transformation

Log transformation

- Compresses large values and makes relationships between variables easier for a model to capture
- Useful when dealing with heavily skewed data
- log is not defined for zeros or negative values



©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

How to implement

Standardization, normalization, log transformation

```
import numpy as np

# standardization
from sklearn.preprocessing import StandardScaler
ss = StandardScaler()
x_std = ss.fit_transform(x)

# normalization
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
x_normalized = scaler.fit_transform(x)

# log transform
x_log_trans = np.log(x)
```

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

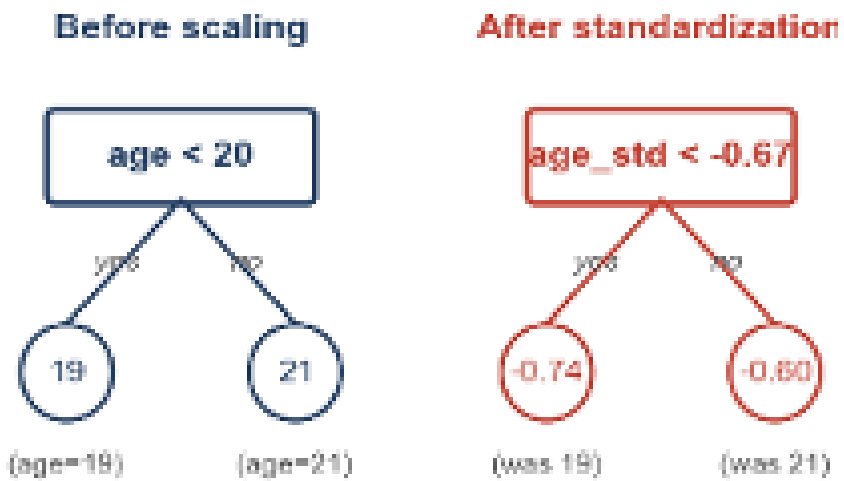
Scaling has (almost) no effect on tree-based models

Standardization, normalization, log transformation

- Trees split on rank order — not absolute scale

Decision Tree / RF / XGBoost / LightGBM: results essentially identical

Still useful for: linear regression, logistic regression, k-NN, SVM



©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming Categorical Variables

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming Categories (1): Label Encoding

Label Encoding

Assign each category an integer (e.g. red→1, green→2, blue→3)

Color
Red
Green
Blue
Green
Green
Red

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming Categories (1): Label Encoding

Label Encoding

Assign each category an integer (e.g. red→1, green→2, blue→3)

Color
Red
Green
Blue
Green
Green
Red



Color
1
2
3
2
2
1

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming Categories (1): Label Encoding

Label Encoding

Assign each category an integer (e.g. red→1, green→2, blue→3)

- Merits: Simple, Works fine with tree-based models
- Demerits: introduces a fake ordering

Color
Red
Green
Blue
Green
Green
Red



Color
1
2
3
2
2
1

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming Categories (2): One-hot Encoding

One-hot Encoding

Create a separate column for each category and assign 1 to the matched category

Color
Red
Green
Blue
Green
Green
Red

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming Categories (2): One-hot Encoding

One-hot Encoding

Create a separate column for each category and assign 1 to the matched category

Color
Red
Green
Blue
Green
Green
Red



color_red	color_green	color_blue

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming Categories (2): One-hot Encoding

One-hot Encoding

Create a separate column for each category and assign 1 to the matched category

Color
Red
Green
Blue
Green
Green
Red



color_red	color_green	color_blue
1	0	0

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming Categories (2): One-hot Encoding

One-hot Encoding

Create a separate column for each category and assign 1 to the matched category

Color
Red
Green
Blue
Green
Green
Red



color_red	color_green	color_blue
1	0	0
0	1	0
0	0	1

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming Categories (2): One-hot Encoding

One-hot Encoding

Create a separate column for each category and assign 1 to the matched category

Color
Red
Green
Blue
Green
Green
Red



color_red	color_green	color_blue
1	0	0
0	1	0
0	0	1
0	1	0
0	1	0
1	0	0

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Transforming Categories (2): One-hot Encoding

One-hot Encoding

Create a separate column for each category and assign 1 to the matched category

- Merits: removes numerical relationships
- Demerits: high memory consumption

Color
Red
Green
Blue
Green
Green
Red



color_red	color_green	color_blue
1	0	0
0	1	0
0	0	1
0	1	0
0	1	0
1	0	0

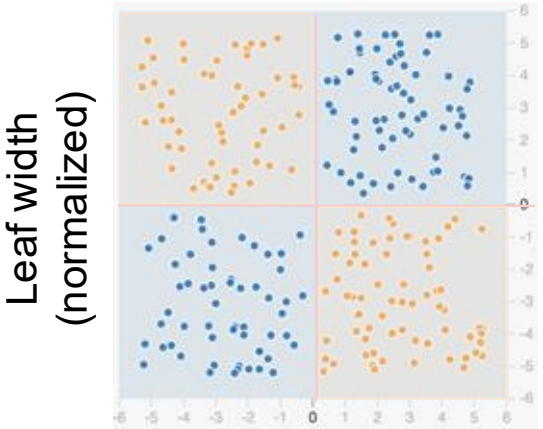
©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Cross Features

Cross features

A new feature built from a product of 2+ existing ones

- a, b, c → a×b, a×c, b×c, a×b×c …
- e.g., add (room size) × (distance to station) in rent prediction



Length	Width	Class
1	1	0
3	4	0
-3	-2	0
-2	1	1
4	-4	1

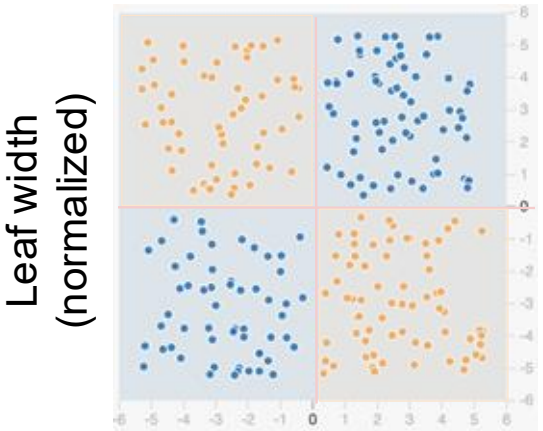
©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Cross Features

Cross features

A new feature built from a product of 2+ existing ones

- a, b, c → a×b, a×c, b×c, a×b×c …
- e.g., add (room size) × (distance to station) in rent prediction



Length	Width	Class
1	1	0
3	4	0
-3	-2	0
-2	1	1
4	-4	1

Length x Width	Class
1	0
12	0
6	0
-2	1
-16	1

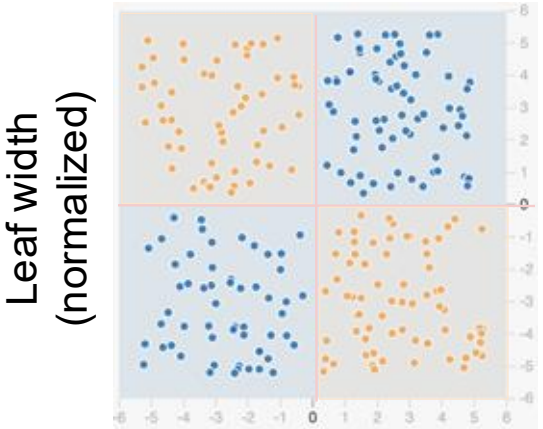
©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Cross Features

Cross features

A new feature built from a product of 2+ existing ones

- a, b, c → a×b, a×c, b×c, a×b×c …
- e.g., add (room size) × (distance to station) in rent prediction



Length	Width	Class
1	1	0
3	4	0
-3	-2	0
-2	1	1
4	-4	1

Length x Width	Class
1	0
12	0
6	0
-2	1
-16	1

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Cross Features

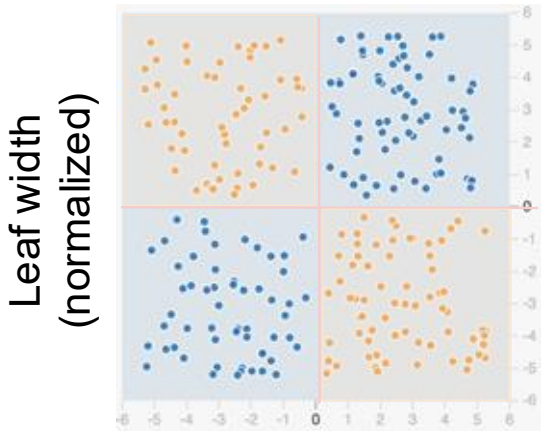
Cross features

A new feature built from a product of 2+ existing ones

- a, b, c → a×b, a×c, b×c, a×b×c …
- e.g., add (room size) × (distance to station) in rent prediction

Merits: captures interactions the model cannot easily learn on its own

Demerits: memory consumption, overfitting risk



Length	Width	Class
1	1	0
3	4	0
-3	-2	0
-2	1	1
4	-4	1

Length x Width	Class
1	0
12	0
6	0
-2	1
-16	1

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

How to implement

Label / one-hot encoding & cross features

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# sample categorical data
df = pd.DataFrame({
    "city": ["Tokyo", "Bangkok", "Tokyo", "Jakarta"],
    "device": ["mobile", "pc", "tablet", "mobile"]
})

# label encoding
df["city_label"] = LabelEncoder().fit_transform(df["city"])

# one-hot encoding
df_onehot = pd.get_dummies(df, columns=["city", "device"])
```

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

How to implement

Label / one-hot encoding & cross features

```
import pandas as pd

# sample categorical data
df = pd.DataFrame({
    "city": ["Tokyo", "Bangkok", "Tokyo", "Jakarta"],
    "device": ["mobile", "pc", "tablet", "mobile"]
})

# cross feature
df["city_device"] = df["city"] + "_" + df["device"]
df_cross = pd.get_dummies(df, columns=["city_device"])
```

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Use of Domain Knowledge

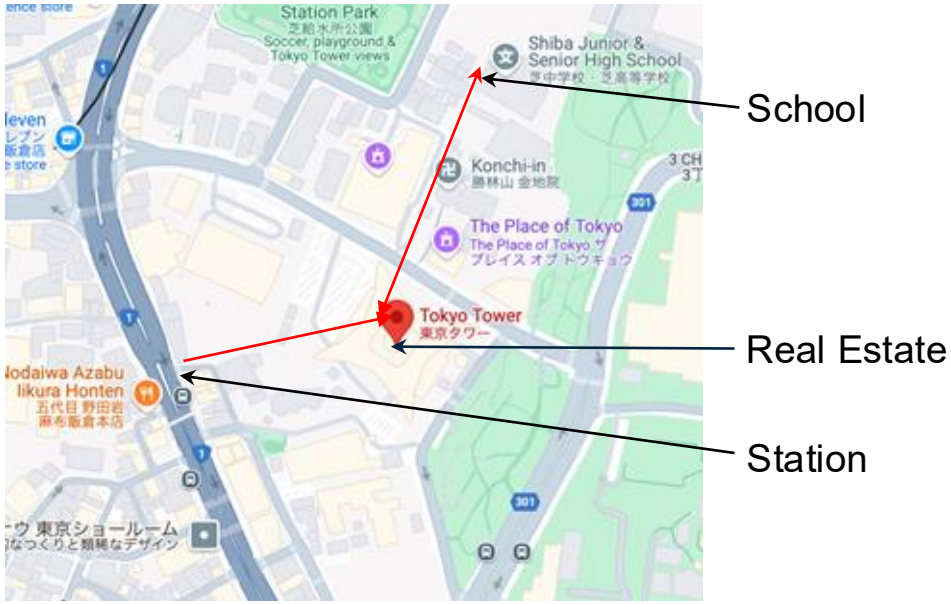
©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Domain Knowledge for Feature Engineering

Domain knowledge (expertise in a specific field) helps us transform raw data into meaningful features that are more directly related to the prediction target.

e.g., Rent prediction

- Raw data
  - address, size, age of building



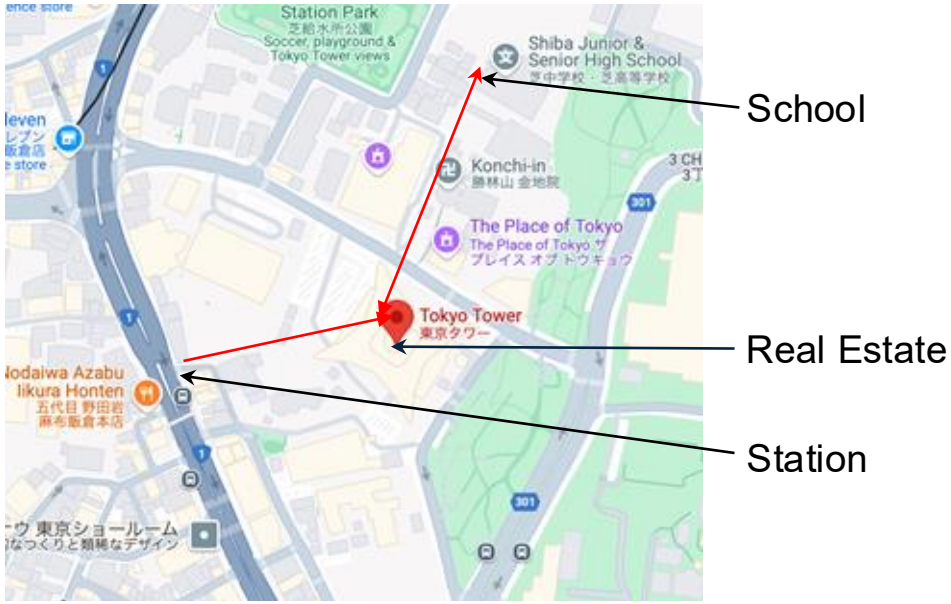
©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Domain Knowledge for Feature Engineering

Domain knowledge (expertise in a specific field) helps us transform raw data into meaningful features that are more directly related to the prediction target.

e.g., Rent prediction

- Raw data
  - address, size, age of building
- Using domain knowledge, create features like
  - Distance to nearest station
  - Distance to nearest school



©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Domain Knowledge for Feature Engineering

Domain knowledge (expertise in a specific field) helps us transform raw data into meaningful features that are more directly related to the prediction target.

Domain/task	Raw data	Domain knowledge-based features
🏠 Real estate: predict house prices	Address, size, age of building	Distance to nearest station, price per square meter, school district

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Domain Knowledge for Feature Engineering

Domain knowledge (expertise in a specific field) helps us transform raw data into meaningful features that are more directly related to the prediction target.

Domain/task	Raw data	Domain knowledge-based features
🏠 Real estate: predict house prices	Address, size, age of building	Distance to nearest station, price per square meter, school district
⚽ Soccer: predict whether a shot becomes a goal	Ball position, player positions	Distance to goal, shooting angle, number of defenders

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Domain Knowledge for Feature Engineering

Domain knowledge (expertise in a specific field) helps us transform raw data into meaningful features that are more directly related to the prediction target.

Domain/task	Raw data	Domain knowledge-based features
🏠 Real estate: predict house prices	Address, size, age of building	Distance to nearest station, price per square meter, school district
⚽ Soccer: predict whether a shot becomes a goal	Ball position, player positions	Distance to goal, shooting angle, number of defenders
🛍️ E-commerce: predict whether a user buys a product	Browsing history, purchase history	Number of views in the past week, time since last purchase

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Ensembling

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Ensembling

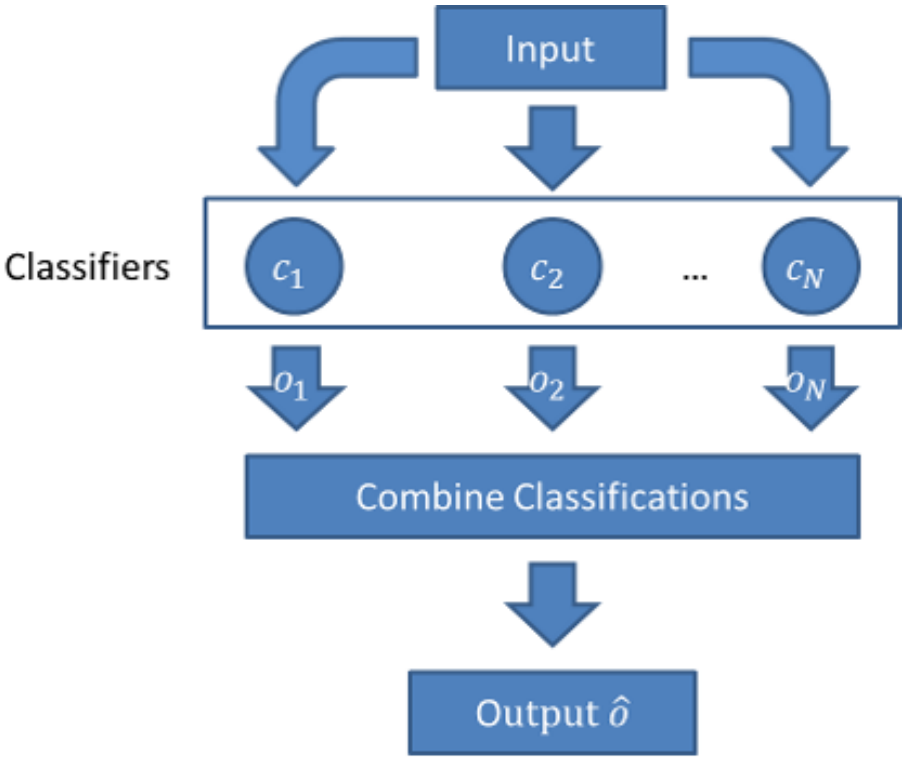
Ensemble learning

- Method to **improve your model performance**

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Ensemble learning

- Method to **improve your model performance**
- Idea: prepare **multiple models**, then **aggregate result** by majority vote or average
  - Majority tends to beat any single model

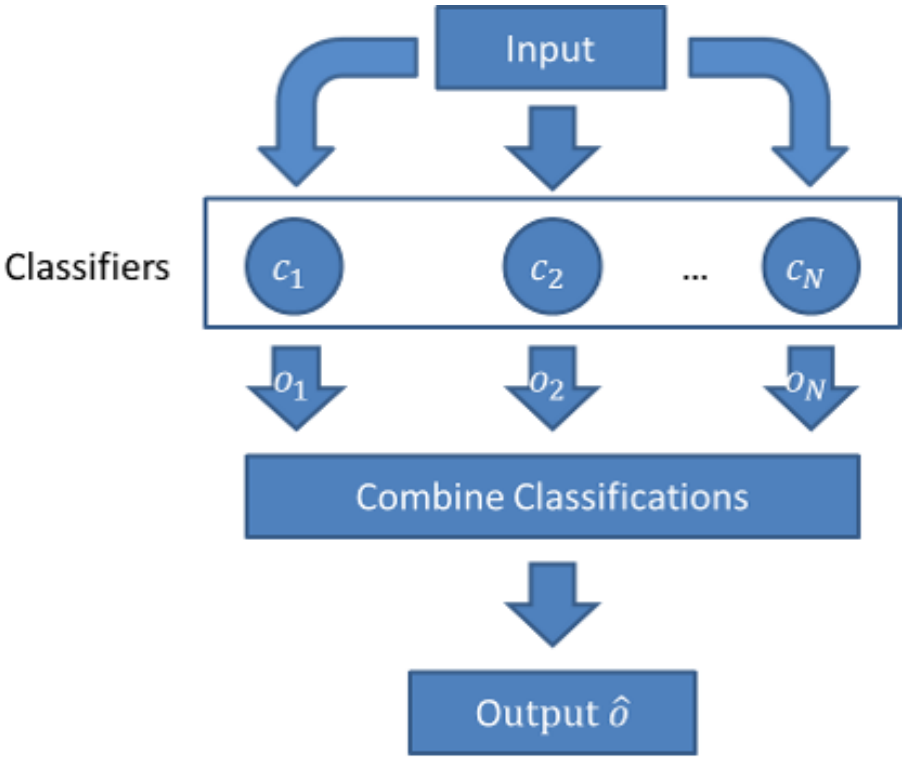


Reference:
https://qiita.com/nirperm/items/318d7e210c059373f8d2

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Ensemble learning

- Method to **improve your model performance**
- Idea: prepare **multiple models**, then **aggregate result** by majority vote or average
  - Majority tends to beat any single model
- Two main families: **Bagging** and **Boosting**



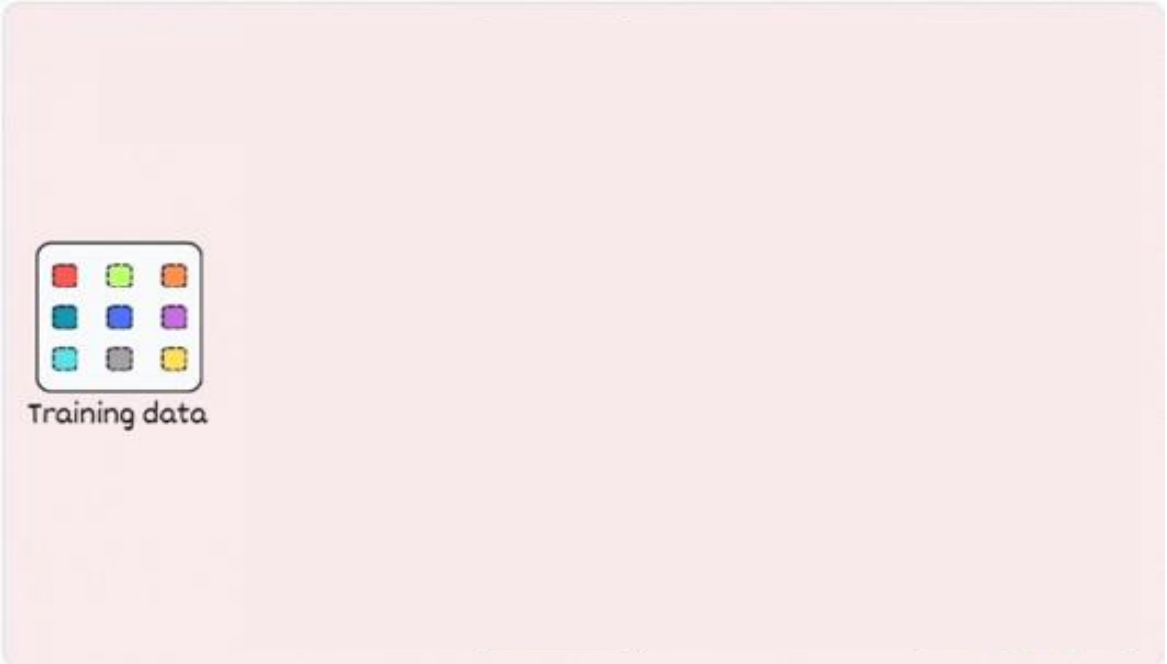
Reference:
https://qiita.com/nirperm/items/318d7e210c059373f8d2

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Bagging (bootstrap aggregating)

Bagging

- Generate **bootstrap resamples** of training data

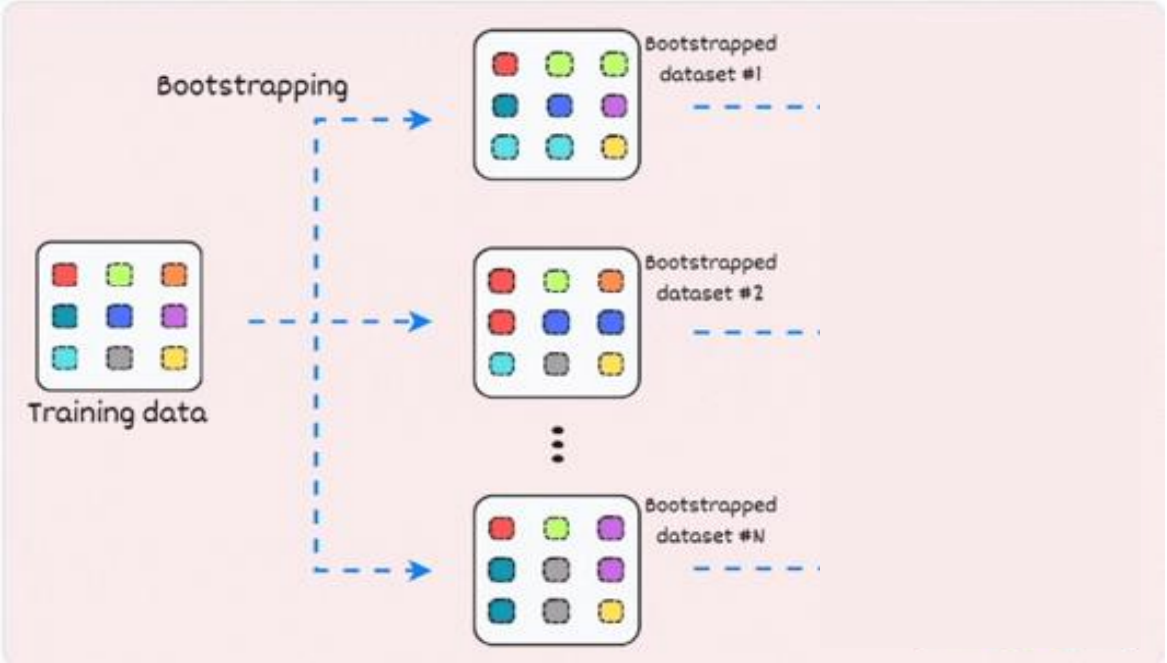


©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Bagging (bootstrap aggregating)

Bagging

- Generate **bootstrap resamples** of training data

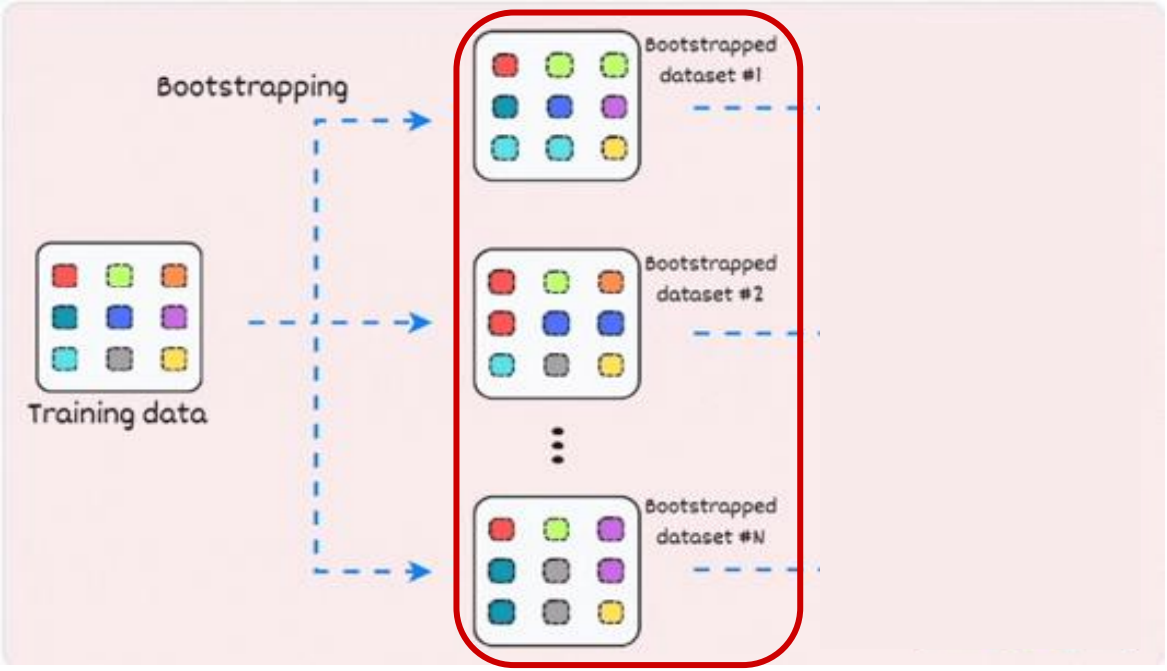


©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Bagging (bootstrap aggregating)

Bagging

- Generate **bootstrap resamples** of training data

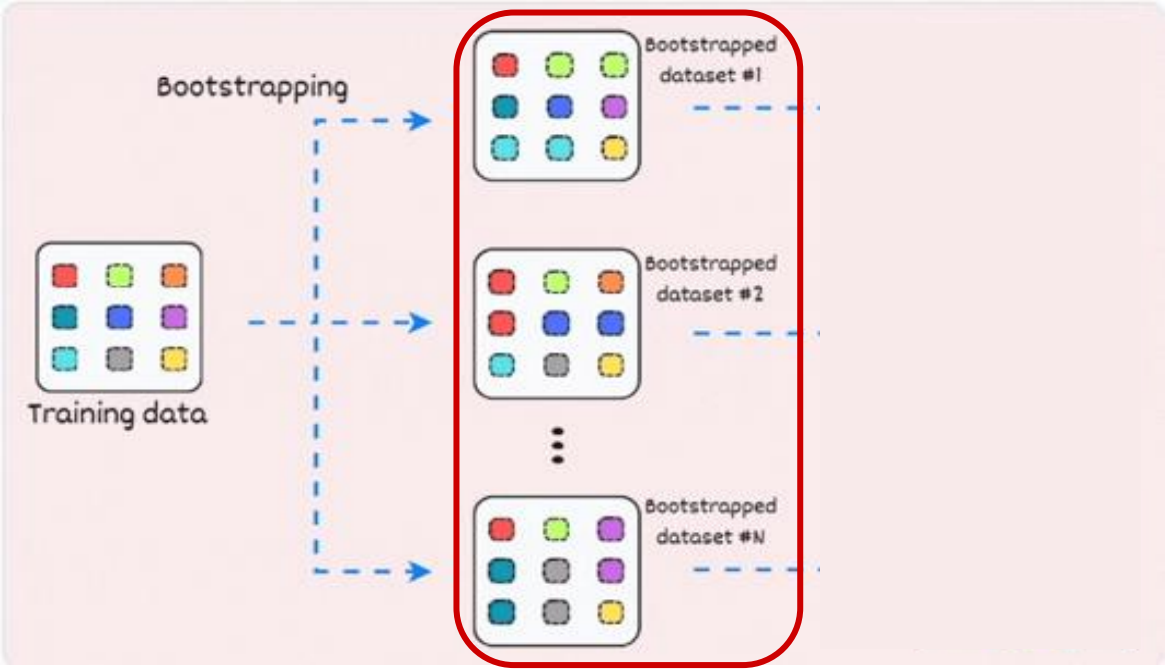


©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Bagging (bootstrap aggregating)

Bagging

- Generate **bootstrap resamples** of training data
  - Bootstrap: to sample from the set with duplicates allowed

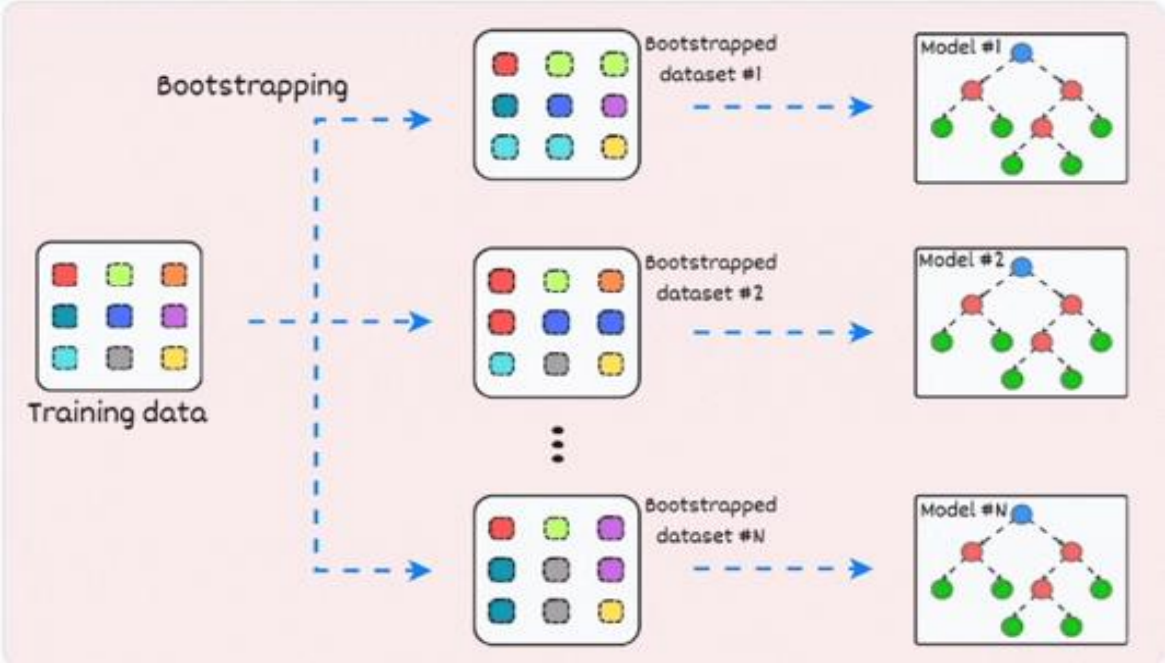


©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Bagging (bootstrap aggregating)

Bagging

- Generate bootstrap resamples of training data
  - Bootstrap: to sample from the set with duplicates allowed
- Train separate model with each resample, then aggregate result
  - Each model may be weaker, but the combined model can be stronger

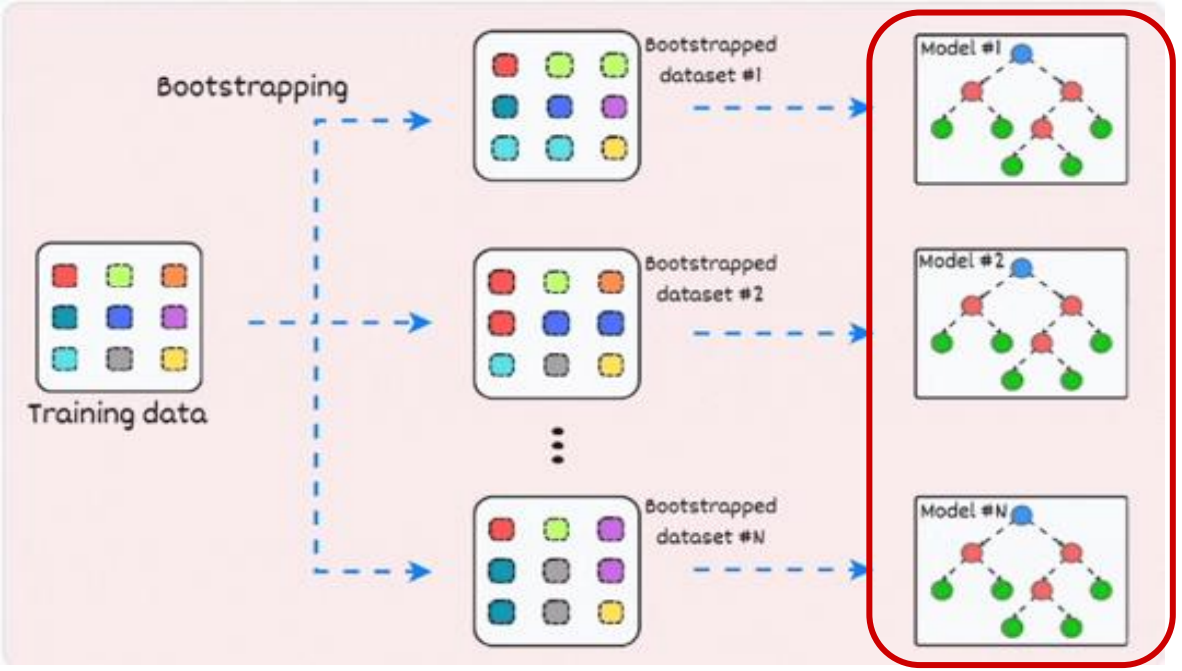


©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Bagging (bootstrap aggregating)

Bagging

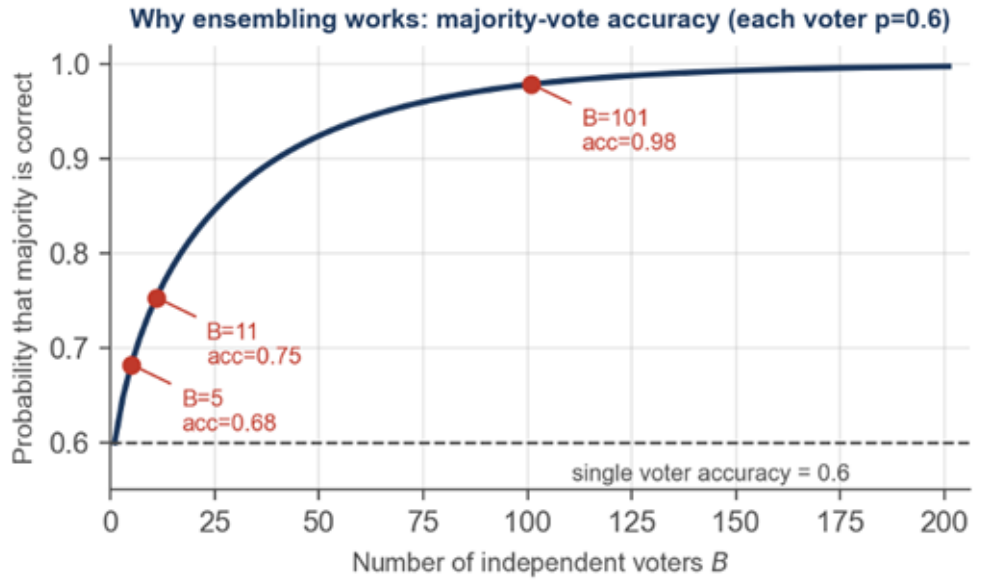
- Generate bootstrap resamples of training data
  - Bootstrap: to sample from the set with duplicates allowed
- Train separate model with each resample, then aggregate result
  - Each model may be weaker, but the combined model can be stronger



©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Why does bagging work?

100 imperfect models can outperform 1 strong model — if mistakes are diverse.

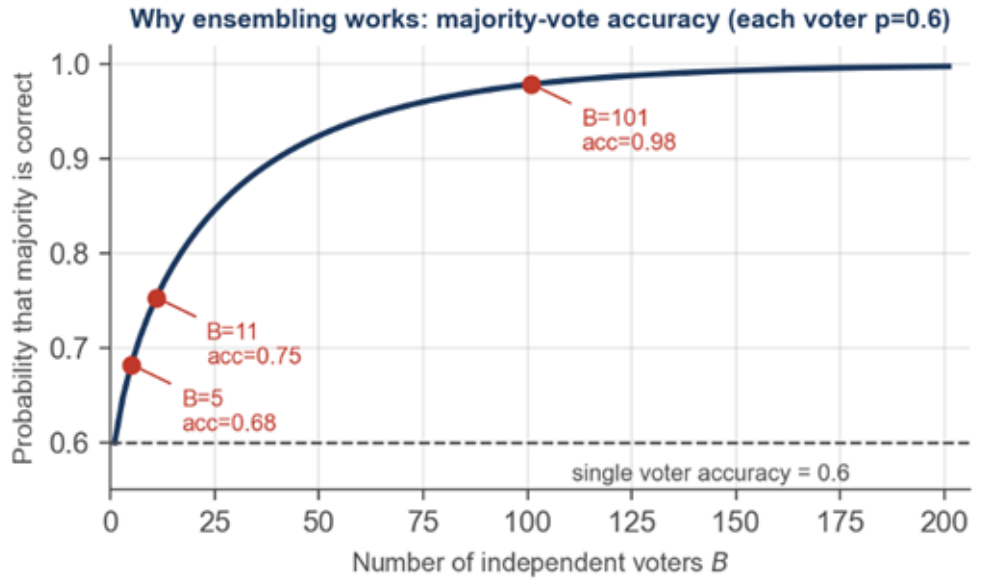


©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Why does bagging work?

100 imperfect models can outperform 1 strong model — if mistakes are diverse.

- 1 expert (60% correct) → 60% accuracy

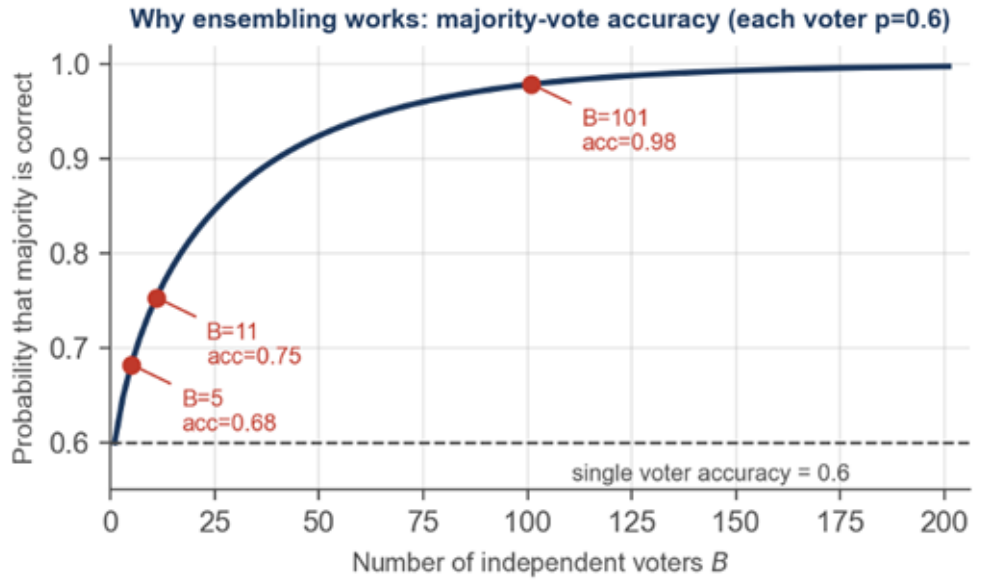


©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Why does bagging work?

100 imperfect models can outperform 1 strong model — if mistakes are diverse.

- 1 expert (60% correct) → 60% accuracy
- 5 independent voters → ~68%

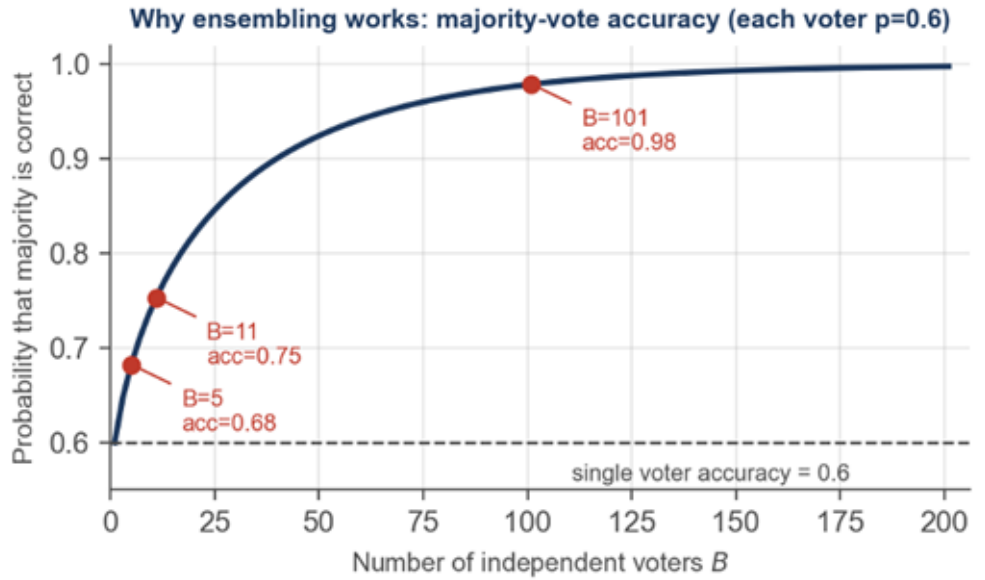


©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Why does bagging work?

100 imperfect models can outperform 1 strong model — if mistakes are diverse.

- 1 expert (60% correct) → 60% accuracy
- 5 independent voters → ~68%
- 11 voters → ~75%
- 101 voters → ~98%

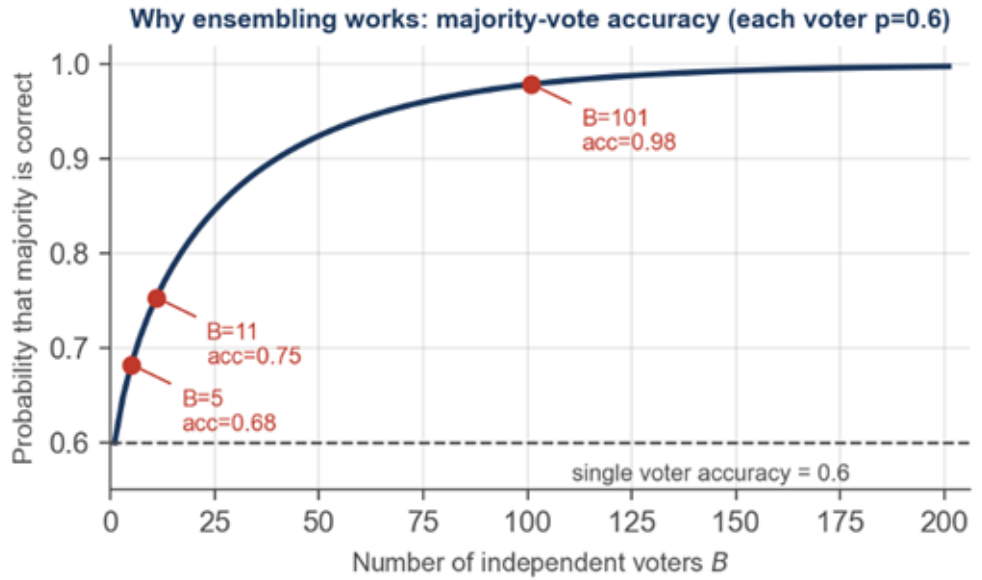


©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Why does bagging work?

100 imperfect models can outperform 1 strong model — if mistakes are diverse.

- 1 expert (60% correct) → 60% accuracy
- 5 independent voters → ~68%
- 11 voters → ~75%
- 101 voters → ~98%



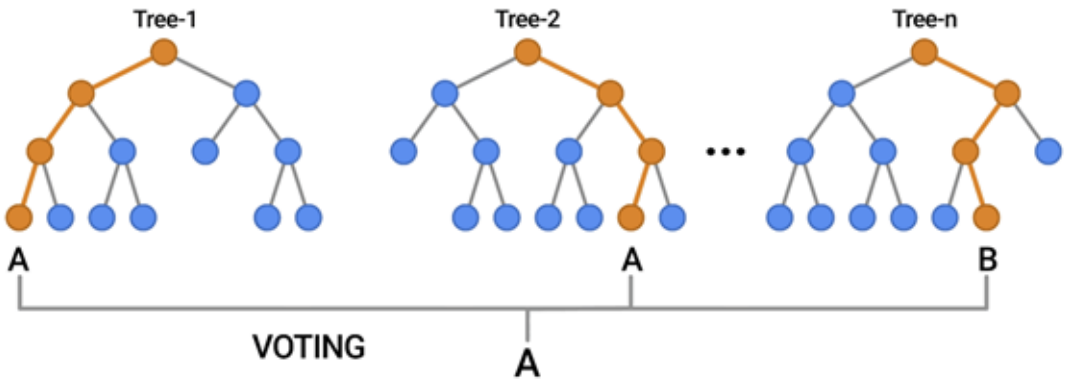
Caveat: if everyone makes the same mistake, voting doesn't work.

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Bagging example: Random forest

Random forest

- Bagging used for decision trees
- Decision trees often look very similar (e.g. one strong feature always used for first split)



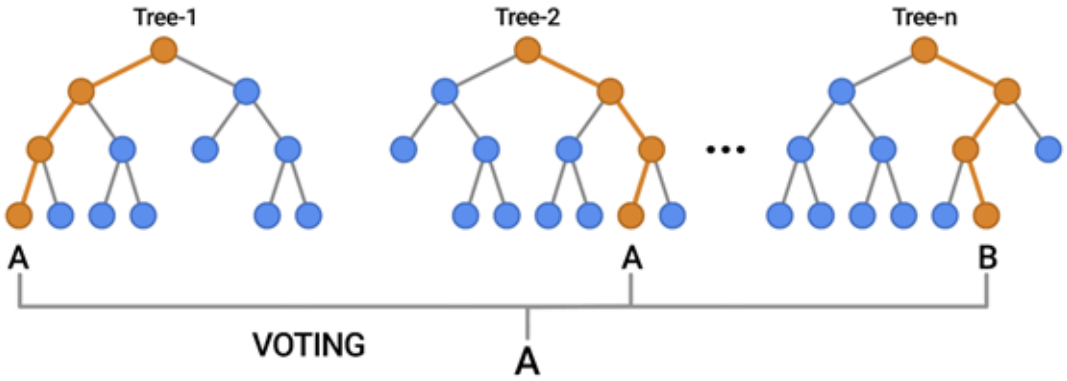
Reference:
https://blog.tensorflow.org/2021/05/introducing-tensorflow-decision-forests.html

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Bagging example: Random forest

Random forest

- Bagging used for decision trees
- Decision trees often look very similar (e.g. one strong feature always used for first split)
- Only use m features to decide each split
- Train many trees, then take the average

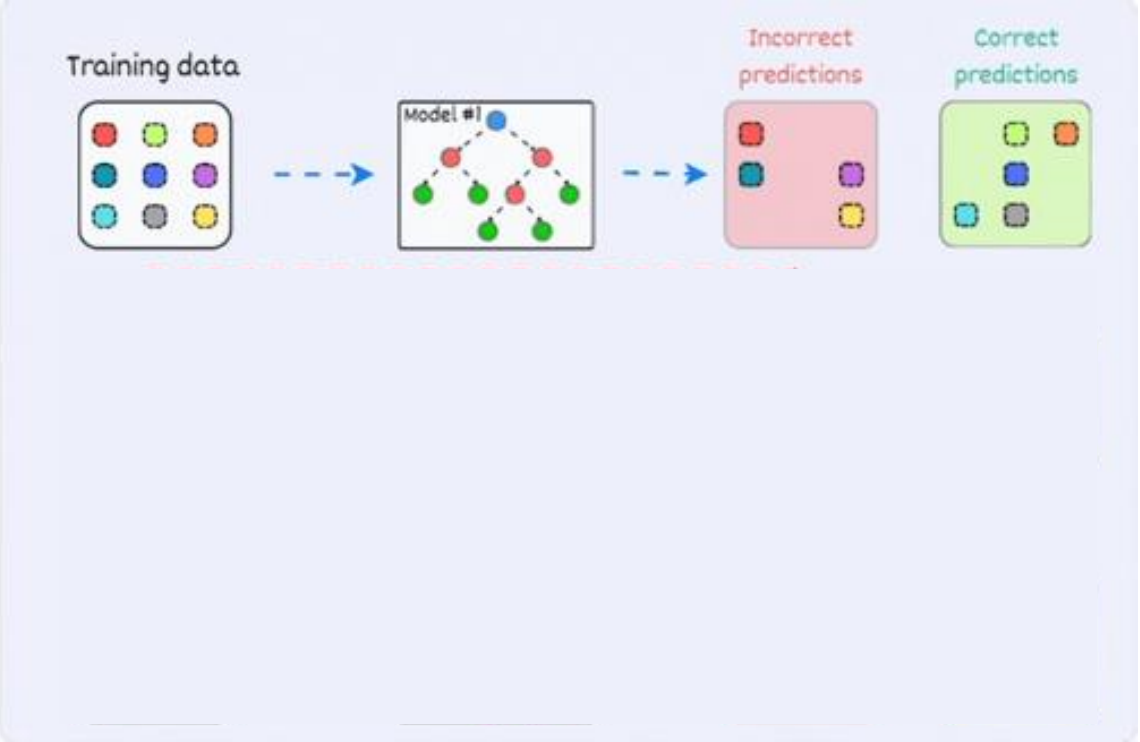


Reference:
https://blog.tensorflow.org/2021/05/introducing-tensorflow-decision-forests.html

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Boosting

- Train a weak model and check **incorrect** predictions

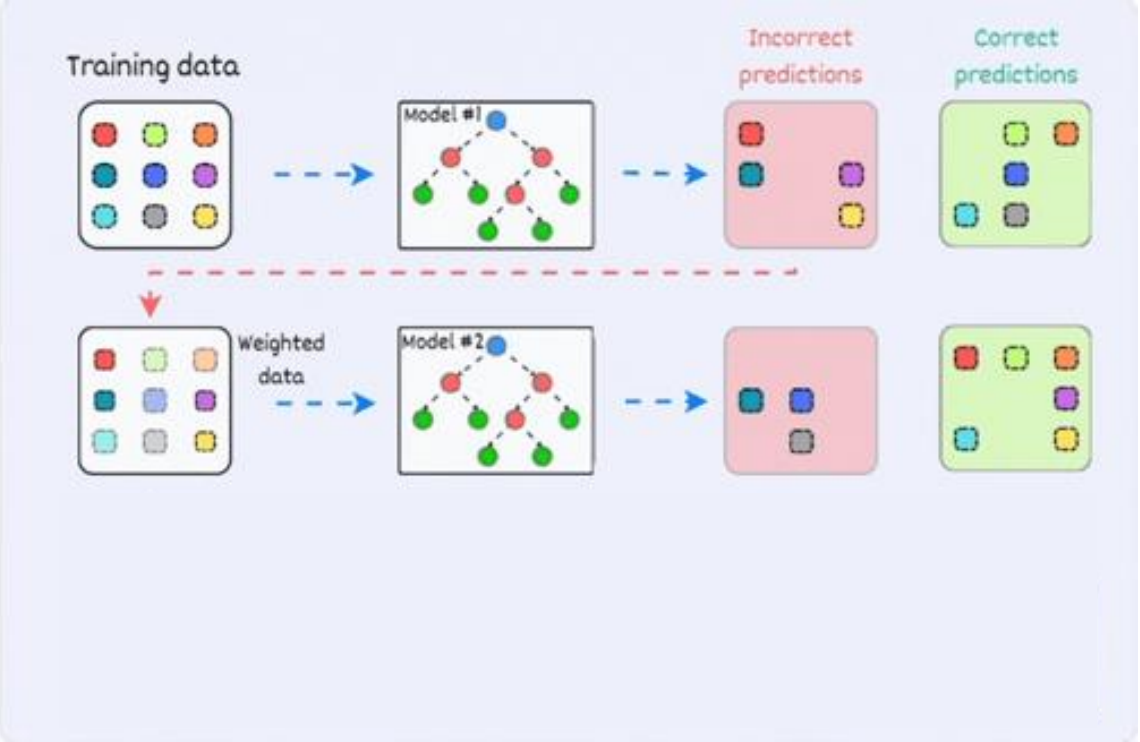


Reference:
https://blog.dailydoseofds.com/p/an-animated-guide-to-bagging-and

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Boosting

- Train a weak model and check incorrect predictions
- Reweight the data, train another model
- Repeat, then take the average

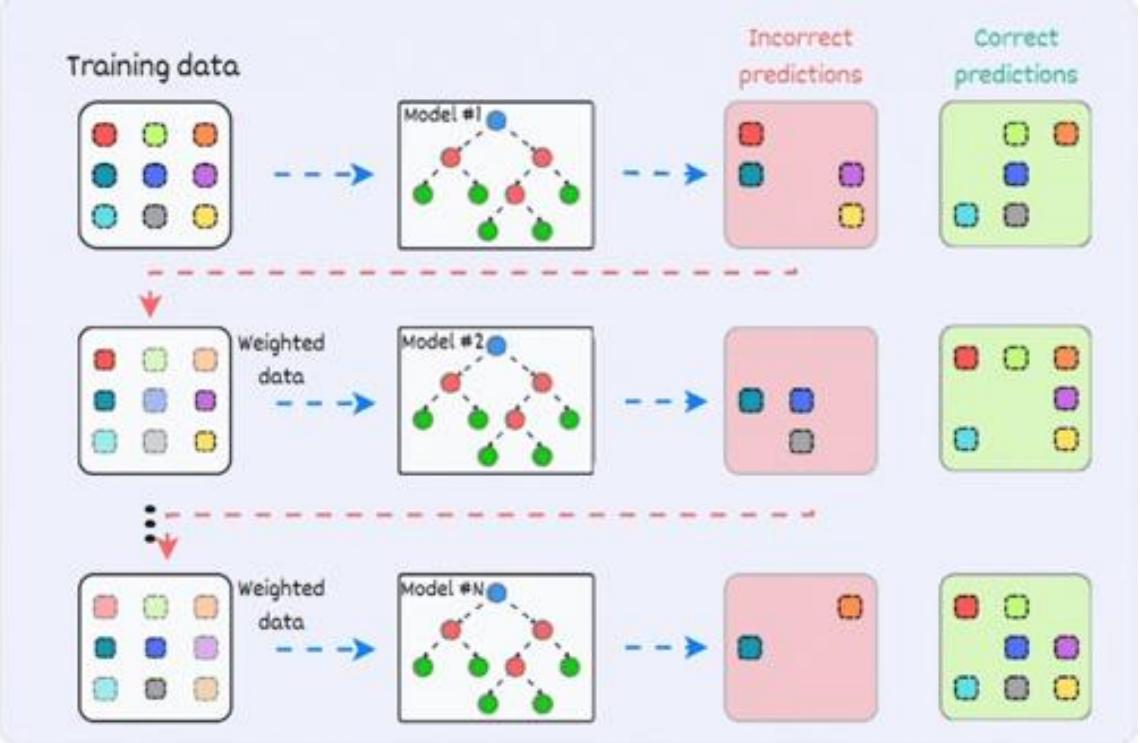


Reference:
https://blog.dailydoseofds.com/p/an-animated-guide-to-bagging-and

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Boosting

- Train a weak model and check incorrect predictions
- Reweight the data, train another model
- Repeat, then take the average

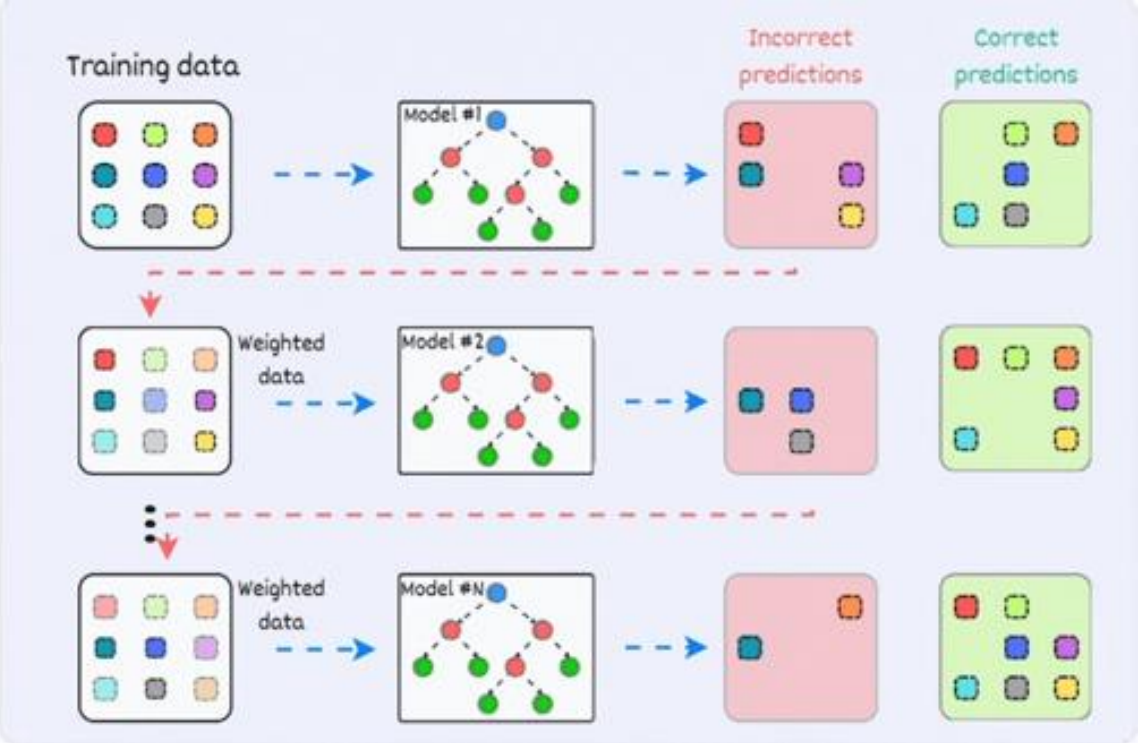


Reference:
https://blog.dailydoseofds.com/p/an-animated-guide-to-bagging-and

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Boosting

- Train a weak model and check incorrect predictions
- Reweight the data, train another model
- Repeat, then take the average
- Examples:
  - Adaboost
  - Gradient boosting
  - XGBoost
  - lightGBM, etc.



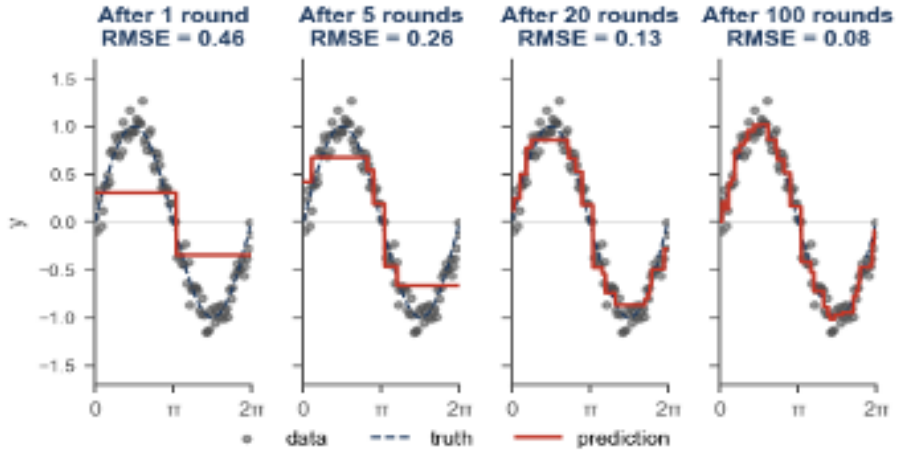
Reference:
https://blog.dailydoseofds.com/p/an-animated-guide-to-bagging-and

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Why does boosting work?

Steps:

- Round 1 captures the easy patterns
- Round 2 fixes what's still wrong

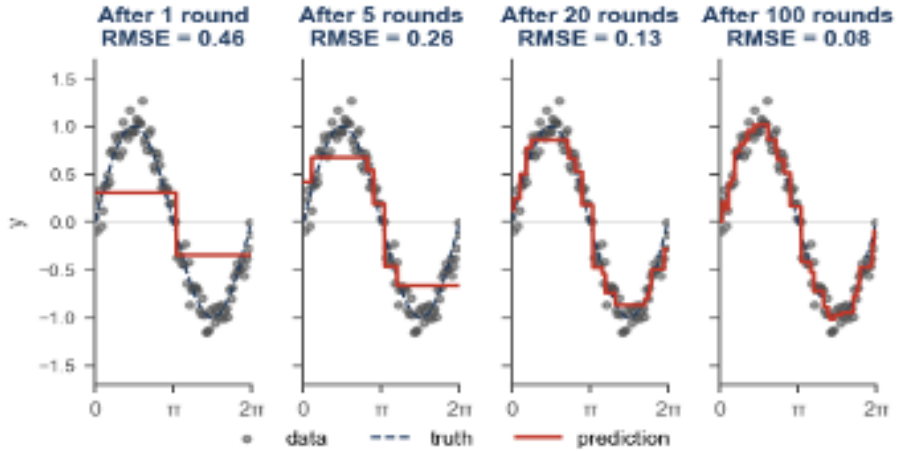


©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Why does boosting work?

Steps:

- Round 1 captures the easy patterns
- Round 2 fixes what's still wrong
- Round 3 fixes what's left after that
- ...



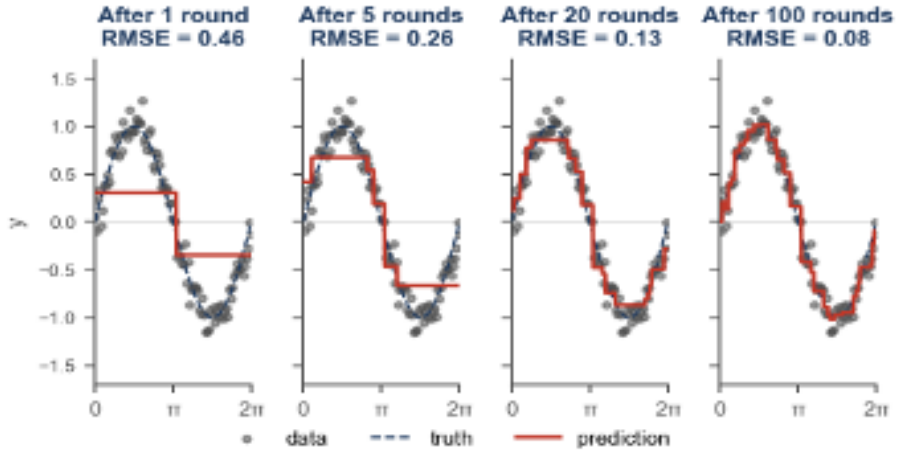
→ Residual (truth − prediction) shrinks every round

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Why does boosting work?

Steps:

- Round 1 captures the easy patterns
- Round 2 fixes what's still wrong
- Round 3 fixes what's left after that
- ...



→ Residual (truth − prediction) shrinks every round

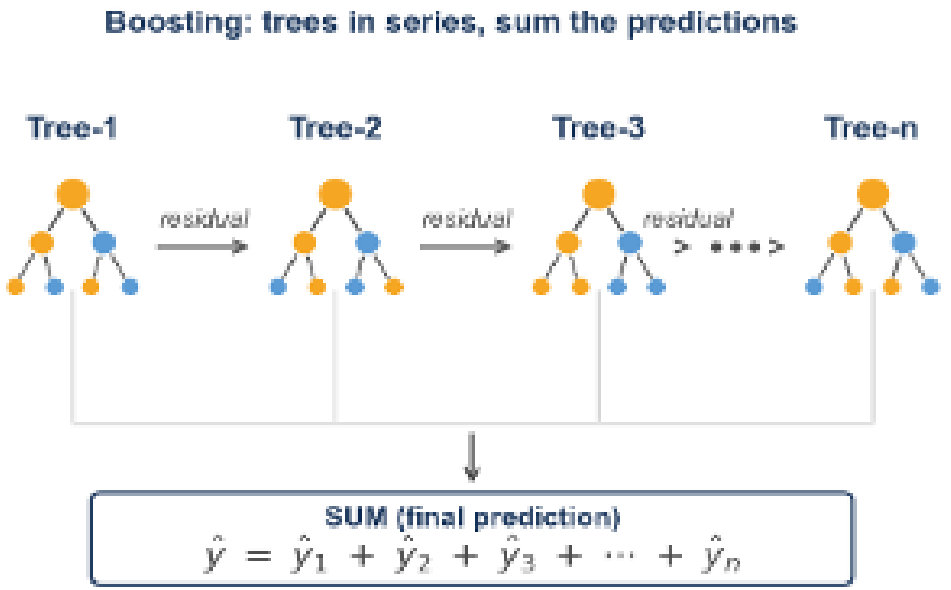
Caveat: try to decrease noises -> overfitting with increased iterations

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Boosting example: XGBoost

XGBoost

- Boosting used for decision trees
- Each tree is shallow (depth ≈ 3–8)
- Each new tree predicts the residual
- Train many trees, then sum (not average)



©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

How to implement

Bagging (Random forest)

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

base_clf = DecisionTreeClassifier(random_state=0)
clf = BaggingClassifier(
    base_estimator=base_clf, # <- classifier you want to apply bagging to
    n_estimators=100,        # number of classifiers
    random_state=0,          # for reproducability
)

clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

How to implement

Gradient boosting

```
from sklearn.ensemble import GradientBoostingRegressor

reg = GradientBoostingRegressor(
    random_state=0
)
reg.fit(X_train, y_train)
y_pred = reg.predict(X_test)
```

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

How to implement

XGboosting

```
from xgboost import XGBRegressor

X_train, X_valid, y_train, y_valid = train_test_split(X, y, test_size=0.2)

reg = XGBRegressor(
    n_estimators=1000,
    eval_metric='rmse',        # evaluation metric
    early_stopping_rounds=20   # finish learning to avoid overfitting
)

reg.fit(X_train, y_train,
eval_set=[(X_valid, y_valid)])
y_pred = reg.predict(X_test)
```

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Section summary

Bagging	RandomForest	When over-learning is to be avoided As a baseline model in the early stages of a project when useful features have not been identified	First option as bagging
	BaggingClassifier /Regressor		In applications where models except decision tree systems are suitable.
Boosting	**XGBoost** GradientBoosting	When you want to pursue accuracy When you have found useful features in EDA to some extent, etc.	First choice in many cases.
	AdaBoost		In applications where models except decision tree systems are suitable.
Ensemble not used	Stand-alone model	When interpretability of results matters	

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Key Takeaways

- Feature engineering can impact your model's performance
  - Feature transformation: standardization, normalization, etc.
  - Feature creation: combine features, use domain knowledge, etc.

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Key Takeaways

- Feature engineering can impact your model's performance
  - Feature transformation: standardization, normalization, etc.
  - Feature creation: combine features, use domain knowledge, etc.

- Feature engineering is iterative
  - Need try-and-error loops

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Key Takeaways

- Feature engineering can impact your model's performance
  - Feature transformation: standardization, normalization, etc.
  - Feature creation: combine features, use domain knowledge, etc.

- Feature engineering is iterative
  - Need try-and-error loops

- Ensembling is very useful in real-world applications
  - Bagging: random forest
  - Boosting: AdaBoost, XGBoost, LightGBM, etc.

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

松尾・岩澤研究室
MATSUO-IWASAWA LAB UTOKYO

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

How to implement

XGboosting

```
from xgboost import XGBRegressor

X_train, X_valid, y_train, y_valid = train_test_split(X, y, test_size=0.2)

reg = XGBRegressor(
    n_estimators=1000,
    eval_metric='rmse',        # evaluation metric
    early_stopping_rounds=20   # finish learning to avoid overfitting
)

reg.fit(X_train, y_train,
eval_set=[(X_valid, y_valid)])
y_pred = reg.predict(X_test)
```

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Creating features: Domain Knowledge

Domain knowledge leads to better features



→ Generate ‘distance_school’, ‘distance_station’

anti_a_rbc	anti_b_rbc	anti_a_plasma	anti_b_plasma	anti_o_plasma
+	-	-	+	-
+	+	-	-	-
-	+	+	-	-
-	-	+	+	-

Blood Type
⟶ A
⟶ AB

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Creating features: Combine Features

Effective especially for the simpler algorithms that might not uncover complex interactions automatically

2012-12-01	119.0	NaN	NaN
2012-12-02	313.0	119.0	2.630252
2012-12-03	40.0	313.0	0.127796
2012-12-04	39.0	40.0	0.975000
2012-12-05	29.0	39.0	0.743590

← 313.0 ÷ 119.0

← 40.0 ÷ 313.0

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut commodo mauris nunc, quis scelerisque urna tristique et. Nullam odio lorem, maximus at vehicula a, porta ac libero. Donec tempus tellus eu lacus interdum, in bibendum enim mattis. Pellentesque et tempus ligula. Sed ex ligula, ullamcorper ut eros quis, elementum placerat magna. Curabitur vel viverra ipsum. Suspendisse aliquet nisi ante, non porta metus scelerisque quis.

lorum ×2 ipsum ×2

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

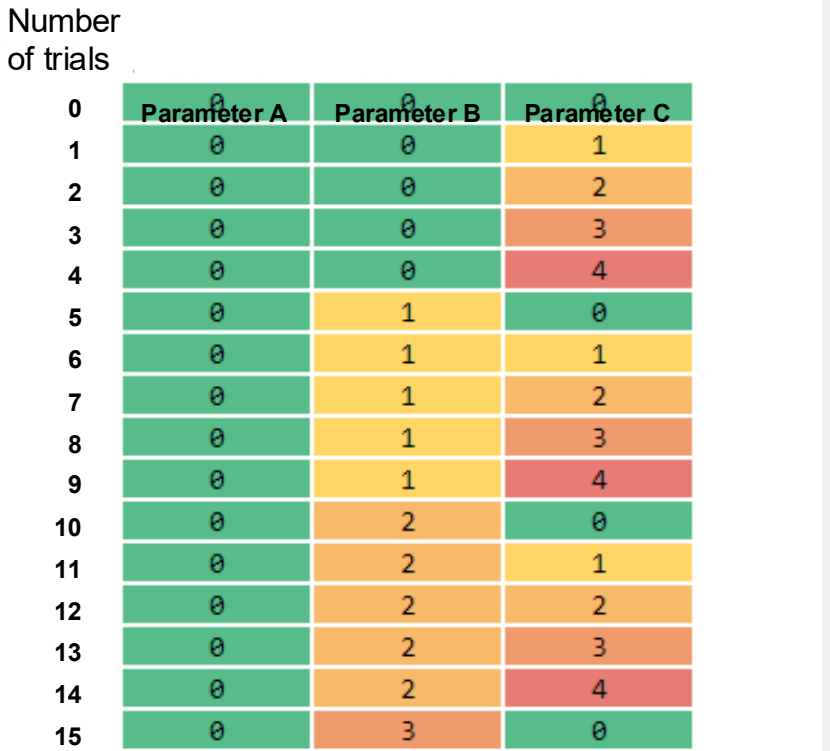
Hyperparameter Search (1) : Grid Search

Characteristics of This Search Method

Exhaustive search through all combinations

- Advantage: No duplicates or omissions.
- Disadvantage: The computational load increases significantly with the number of hyperparameter combinations, leading to:↓
  - Longer execution times
  - Limited ability to explore a wide range of combinations.

Not suitable for initial exploration.
Best suited for fine-tuning.



Grid search is a method that systematically explores all combinations one by one.

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Classification Model Evaluation Metrics: A Summary for Practical Use

Recall	• When **missed detections** are significantly higher risk than false positives (e.g., cancer screening, malware detection, disaster prediction, child abuse detection).
Precision	• When **false positives** are significantly higher risk than missed detections (e.g., spam, detection, decisions for irreversible procedures, purchasing speculative financial products).
F-Value(F1)	• When comparing models using a **comprehensive evaluation** and **Recall** and **Precision.**
ROC-AUC	• When comparing models with **imbalanced data** and evaluating overall performance across all thresholds. • When comparing across **different problem settings.**
Accuracy	• When you want a **quick, intuitive** accuracy assessment with balanced data. • When explaining complex evaluation metrics to a novice.

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

Hyperparameters

A set of parameters of the machine learning flow that needs to be set by humans

↔ c.f. parameters: values & settings that are internal to the model

Examples:

Decision Tree

```
model = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=5,
    random_state=0
)
```

Ridge Regression

```
ridge = Ridge(
    alpha = 0.1
)
```

$$J(w) = \sum_{i=1}^{n}(y_i - f(x_i))^2 + \alpha \sum_{j=1}^{p} w_j^2$$

Lasso Regression

```
Lasso = Lasso(
    alpha = 0.1
)
```

$$J(w) = \sum_{i=1}^{n}(y_i - f(x_i))^2 + \alpha \sum_{j=1}^{p} |w_j|$$

*If not specified, it will set to default value (see documentation)

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO